

2025 年度人工知能特論レポート

M1

清水 透真

学籍番号 253426016

問題 1

東証プライムに上場している企業を 1 つ選択し、その企業の株価の一日先のリターンが TOPIX のリターンを上回るかどうかについて、複数の教師あり学習手法を用いて予測モデルを構築せよ。検証の際には、訓練データは 2024 年最終日までのデータ、検証データは 2025 年のデータを使用し、予測モデルの性能を評価してその考察をせよ。なお、証券コードは東証上場会社情報サービスが利用できる。また Yahoo! Finance からのオンラインデータでのデータ取得を行う Python のスクリプトについては、report_01.py を参考にせよ。

1. はじめに

東証プライムに上場している企業の中で株式会社 DeNA(証券コード: 2432.T)を選択した。データ取得期間は 2020/07/25~2025/07/25 までとし、dropna()method で欠損値を除いた結果、総データ数は 1224 日となった。ベンチマークとして使用した TOPIX についても、同様に 1224 日のデータ数を扱った。

```
data = yf.download(symbols, period='5y', interval = "1d")['Close']  
[*****100%*****] 2 of 2 completed  
(1224, 2)
```

図 1: 学習データ数()

2. モデルの訓練・検証

2020/07/22~2024/12/31 までのデータを学習用、2025/01/01~2025/07/22 までのデータを検証用とした。データクレンジングや拡張処理は行わず、取得した生データのみを使用した。多クラス分類による学習も検討したが、閾値設定が主観的になりやすいこと、またリターン差の分布が小さく、クラス間の差別化が困難であることから、DeNA が TOPIX のリターンを上回った場合を 1、下回った場合を 0 とする 2 値分類で訓練を行った。以下に使用した特徴量を示す。今回モデルに与えたデータ特徴量は 10 種類である。これらの特徴量をもとにモデルは、短期・中期の値動きの傾向、市場全体との関係、値動きの激しさ、平均からの乖離を学習し、翌日の TOPIX のリターンを上回るかを予測する。

表 1: データ特徴量

target_return_today	対象とした企業の当日と前日の 2 日間における株価の変化率
topix_return_today	TOPIX の当日と前日の 2 日間における株価の変化率
target_return_3d	対象とした企業の当日と 3 日前における株価の変化率
topix_return_3d	TOPIX の当日と 3 日前における株価の変化率
target_rertuen_5d	対象とした企業の当日と 5 日前における株価の変化率
topix_return_5d	TOPIX の当日と 5 日前における株価の変化率
target_vol_5d	対象とした企業の直近 5 日間の値動きのばらつき
topix_vol_5d	TOPIX の直近 5 日間の値動きのばらつき
ma_diff_5d	対象とした企業の直近 5 日間の平均と比べどれだけ高い/低いか
ret_diff	対象企業の値動きが TOPIX の値動きよりどれだけ良い/悪いか

今回使用した教師あり学習手法は、ロジスティック回帰、ランダムフォレスト、SVM、k 近傍法、MLP、XGBoost の 6 種類である。これらの手法によって構築された予測モデルの性能評価には、Accuracy, Precision, Recall, F1 Score を用いた。また、モデルの予測結果を可視化するために混同行列も使用した。以下に作成したプログラムを示す。

```
import yfinance as yf
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score,
f1_score, recall_score, precision_score,
confusion_matrix

target = "2432.T"
symbols = ("1306.T", target)

data = yf.download(symbols, period='5y', interval =
"1d")['Close']
data = data.dropna()

returns = data.pct_change().shift(-1)
returns = returns.dropna()

returns.columns = ['target_return', 'topix_return']

returns['label'] = (returns['target_return'] >
returns['topix_return']).astype(int)

returns['target_return_today'] =
data[target].pct_change()
returns['topix_return_today'] =
data[symbols[0]].pct_change()
returns['target_return_5d'] =
data[target].pct_change(5)
returns['topix_return_5d'] =
data[symbols[0]].pct_change(5)
returns['target_return_3d'] =
data[target].pct_change(3)
returns['topix_return_3d'] =
data[symbols[0]].pct_change(3)
```

```

returns['target_vol_5d'] =
data[target].pct_change().rolling(5).std()
returns['topix_vol_5d'] =
data[symbols[0]].pct_change().rolling(5).std()
returns['ma_diff_5d'] = (data[target] -
data[target].rolling(5).mean()) /
data[target].rolling(5).mean()
returns['ret_diff'] = returns['target_return_today']
- returns['topix_return_today']
returns = returns.dropna()

features = [
    'target_return_today', 'topix_return_today',
    'target_return_3d', 'topix_return_3d',
    'target_return_5d', 'topix_return_5d',
    'target_vol_5d', 'topix_vol_5d',
    'ma_diff_5d', 'ret_diff'
]

train = returns[returns.index < "2025-01-01"]
test = returns[returns.index >= "2025-01-01"]

X_train = train[features]

y_train = train['label']
X_test = test[features]
y_test = test['label']

models = {
    "LogisticRegression":
LogisticRegression(max_iter=1000),
    "RandomForest": RandomForestClassifier(),

```

```

"SVM": SVC(),
"KNN": KNeighborsClassifier(),
"MLP": MLPClassifier(max_iter=1000),
"XGBoost": XGBClassifier(eval_metric='logloss')
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"{name} Accuracy: {accuracy_score(y_test,
y_pred):.3f}")
    print(f"{name} Precision:
{precision_score(y_test, y_pred):.3f}")
    print(f"{name} Recall: {recall_score(y_test,
y_pred):.3f}")
    print(f"{name} F1 Score: {f1_score(y_test,
y_pred):.3f}")
    print(f"{name} Confusion
Matrix:\n{confusion_matrix(y_test, y_pred)}")
    print("-" * 40)

```

3. 結果

教師あり学習によって構築された各予測モデルの結果を以下に示す。

```

LogisticRegression Accuracy: 0.529
LogisticRegression Precision: 0.538
LogisticRegression Recall: 0.792
LogisticRegression F1 Score: 0.640
LogisticRegression Confusion Matrix:
[[15 49]
 [15 57]]

```

```

RandomForest Accuracy: 0.574
RandomForest Precision: 0.595
RandomForest Recall: 0.611
RandomForest F1 Score: 0.603
RandomForest Confusion Matrix:
[[34 30]
 [28 44]]

```

図 2: ロジスティック回帰

図 3: ランダムフォレスト

```
SVM Accuracy: 0.500
SVM Precision: 0.521
SVM Recall: 0.681
SVM F1 Score: 0.590
SVM Confusion Matrix:
[[19 45]
 [23 49]]
```

図 4: SVM

```
KNN Accuracy: 0.500
KNN Precision: 0.526
KNN Recall: 0.569
KNN F1 Score: 0.547
KNN Confusion Matrix:
[[27 37]
 [31 41]]
```

図 5: k 近傍法

```
MLP Accuracy: 0.529
MLP Precision: 0.530
MLP Recall: 0.972
MLP F1 Score: 0.686
MLP Confusion Matrix:
[[ 2 62]
 [ 2 70]]
```

図 6: MLP

```
XGBoost Accuracy: 0.471
XGBoost Precision: 0.500
XGBoost Recall: 0.500
XGBoost F1 Score: 0.500
XGBoost Confusion Matrix:
[[28 36]
 [36 36]]
```

図 7: XGBoost

```
Accuracy: 0.574 (RandomForest)
Precision: 0.595 (RandomForest)
Recall: 0.972 (MLP)
F1: 0.686 (MLP)
```

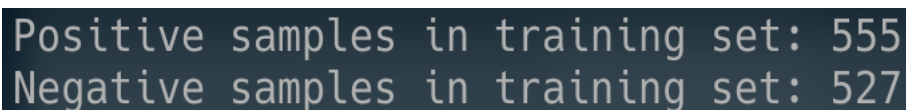
図 8: 各評価指標の最大値

図 8 より、Accuracy および Precision においてはランダムフォレストが最も高い値を示した。一方、Recall および F1 スコアにおいては MLP が最も高い値を示した。混同行列から、MLP はほぼ全てを「DeNA が TOPIX を上回る」と予測する傾向が強いことが分かる。

4. 考察

ランダムフォレストは Accuracy, Precision が最も高く、ノイズを抑制しつつ安定した予測を実現したことが分かる。これは複数の決定木によるアンサンブル学習によって、株価データのような非線形かつノイズを多く含むデータに対応できたと考えられる。ただし、

Recall が 0.500 にとどまり、「DeNA が TOPIX を上回る」の見逃しが一定数発生している。一方、MLP は Recall, F1 スコアが最も高い値を示した。しかし、「ほぼ全てを DeNA が TOPIX を上回る」という極端な予測傾向を示した。これには 2 つの理由があると考えられる。1 つ目として、クラス分布に偏りがあるが挙げられる。2 つ目として、MLP の損失関数の性質が挙げられる。1 つ目のクラス分布の偏りについて、データセットで確認を行った。「DeNA が TOPIX を上回る」は 555 日、「DeNA が TOPIX を下回る」は 527 日と、データ数に差はなかった。データ数に差はないが、データ内の分布(当日と翌日のリターン)に大きな差がなく、モデルが境界を適切に学習できなかったと考える。2 つ目の損失関数の性質については、標準的なクロスエントロピー損失では、出力確率が 0.5 を超えるクラスが 1 として分類されるため、出力確率の分布が偏り、ほぼ全てを「DeNA が TOPIX を上回る」に分類したと考える。



```
Positive samples in training set: 555
Negative samples in training set: 527
```

図 9: クラス分布

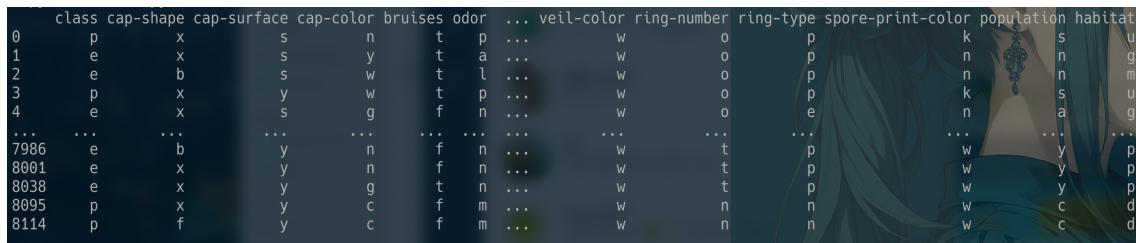
ロジスティック回帰は線形モデルであり、特徴量と目的変数の間に線形関係がある場合、性能を発揮する。しかし、株価データは非線形性が強いいため、ロジスティック回帰では、複雑な関係を捉えきれなかったと考える。SVM は非線形データにも強いモデルであるが、特徴量間のスケーリングや正規化が不十分であり、SVM の距離計算に悪影響を与えたのではないかと考える。XGBoost もまたハイパーパラメータの調整が最適化されていなかったため、精度が低くなったと考える。KNN は、予測時に訓練データとの距離を計算して多数決を取るモデルである。そのため、時系列性や市場構造の非定常性を考慮できない。そのため、他のモデルより精度の低い結果を示したと考える。

問題 2

UCI Machine Learning Repository に Mushroom に関するデータセットがある。このデータは、キノコの形状や生息地から、そのキノコが毒性を持つかどうかに関するものである。実際のデータセットは `agaricus-lepiota.data` に(目的変数は 1 列目)、データそれぞれの説明は `agaricus-lepiota.names` にあるので目を通しておくこと。このデータから、キノコが毒を持つかどうかに関する予測モデルを構築し、PDP, ICE, および SHAP を適用して、予測をどう解釈するかについて考察せよ。

1. はじめに

`agaricus-lepiota.data` には、カラム名が含まれていない。そのため事前にカラムをリストで用意した。カラム名は `agaricus-lepiota.names` を元に定義している。データセット内で欠損値(?)が複数存在しているため、NaN に置換を行い、`dropna()` メソッドで除去をした。欠損値を除去後の総データ数は、8115 個である。



	class	cap-shape	cap-surface	cap-color	bruises	odor	...	veil-color	ring-number	ring-type	spore-print-color	population	habitat
0	p	x	s	n	t	p	...	w	o	p	k	s	u
1	e	x	s	y	t	a	...	w	o	p	n	n	g
2	e	b	s	w	t	l	...	w	o	p	n	n	m
3	p	x	y	w	t	p	...	w	o	p	k	s	u
4	e	x	s	g	f	n	...	w	o	e	n	a	g
...	...	...	...	...	...	...	...	...	...	...	...	...	...
7986	e	b	y	n	f	n	...	w	t	p	w	y	p
8001	e	x	y	n	f	n	...	w	t	p	w	y	p
8038	e	x	y	g	t	n	...	w	t	p	w	y	p
8095	p	x	y	c	f	m	...	w	n	n	w	c	d
8114	p	f	y	c	f	m	...	w	n	n	w	c	d

図 10: データセット

予測対象である `class` 列を目的変数とし、`e`(食用)を 0, `p`(毒)を 1 に変換処理を行った。`class` 列以外の全ての列を説明変数として用いる。今回使用する教師あり学習手法は数値データを入力として必要とするため、カテゴリ変数をダミー変数に変換を行った。

2. モデルの訓練・検証

学習データと検証データを 8:2 に分割を行い、ランダムフォレストに学習させた。以下に

作成したプログラムを示す.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,
accuracy_score
import matplotlib.pyplot as plt
from sklearn.inspection import
PartialDependenceDisplay
import shap

columns = [
    'class', 'cap-shape', 'cap-surface', 'cap-
color', 'bruises', 'odor',
    'gill-attachment', 'gill-spacing', 'gill-size',
    'gill-color',
    'stalk-shape', 'stalk-root', 'stalk-surface-
above-ring',
    'stalk-surface-below-ring', 'stalk-color-above-
ring',
    'stalk-color-below-ring', 'veil-type', 'veil-
color', 'ring-number',
    'ring-type', 'spore-print-color', 'population',
    'habitat'
]

file_path = 'agaricus-lepiota.data'
df = pd.read_csv(file_path, header=None,
names=columns)
```

```
df = df.replace('?', pd.NA)

df = df.dropna()

print(df)

y = df['class'].map({'e': 0, 'p': 1})
X = df.drop('class', axis=1)

X = pd.get_dummies(X)

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)

clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

odor_cols = [col for col in X.columns if
col.startswith('odor_')]

fig, ax = plt.subplots(figsize=(10, 6))
PartialDependenceDisplay.from_estimator(
    clf, X_test, features=odor_cols, kind="both",
ax=ax
)
plt.suptitle('PDP & ICE for odor')
plt.tight_layout()
```

```
plt.show()

# SHAP 値の計算
explainer = shap.TreeExplainer(clf)
shap_values = explainer.shap_values(X_test)
shap_values_class1 = shap_values[:, :, 1]

shap.summary_plot(shap_values_class1, X_test,
plot_type="bar")
shap.summary_plot(shap_values_class1, X_test)
```

3. 結果

特徴量がどのような影響を与えるかを確認するため odor(匂い)に対して PDP & ICE を適用した。また、モデルが予測を行う際どの特徴量を重視したかを確認するため SHAP を適用した。教師あり学習によって構築された予測モデルの結果を以下に示す。

Accuracy: 1.0					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	705	
1	1.00	1.00	1.00	424	
accuracy			1.00	1129	
macro avg	1.00	1.00	1.00	1129	
weighted avg	1.00	1.00	1.00	1129	

図 11: 各評価

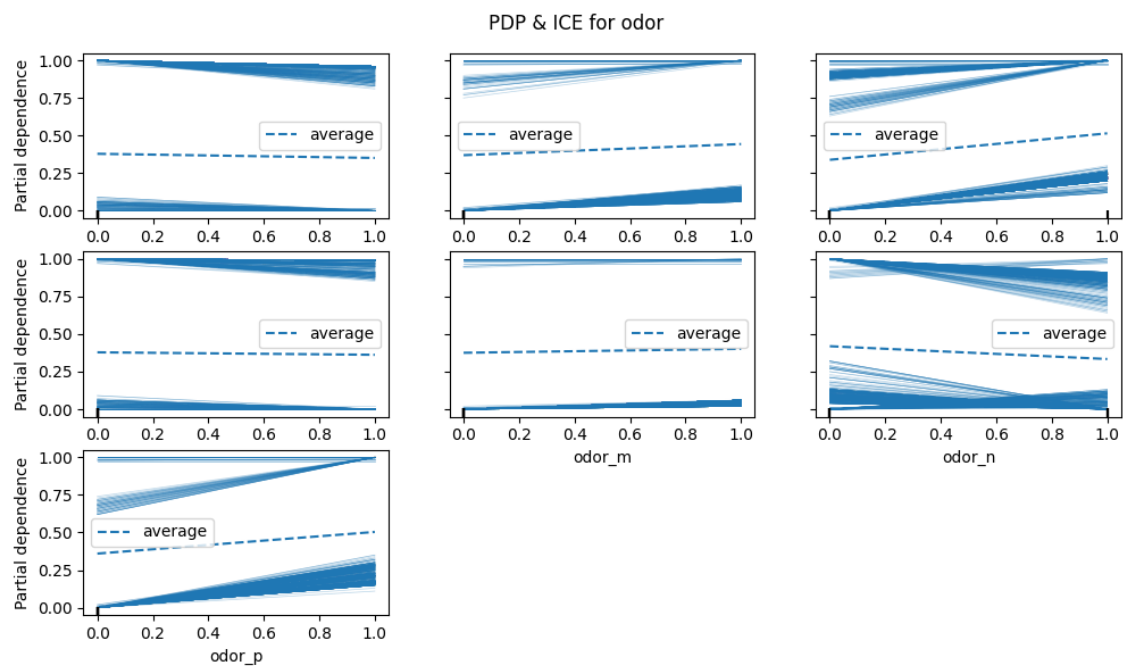


図 12: PDP & ICE

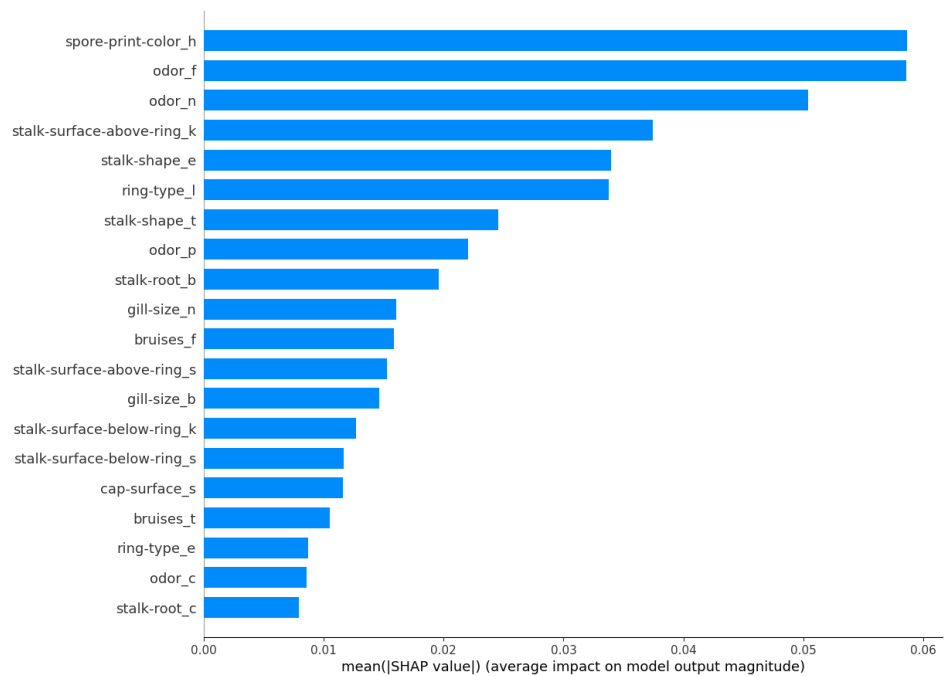


図 13: SHAP(特徴重要度)

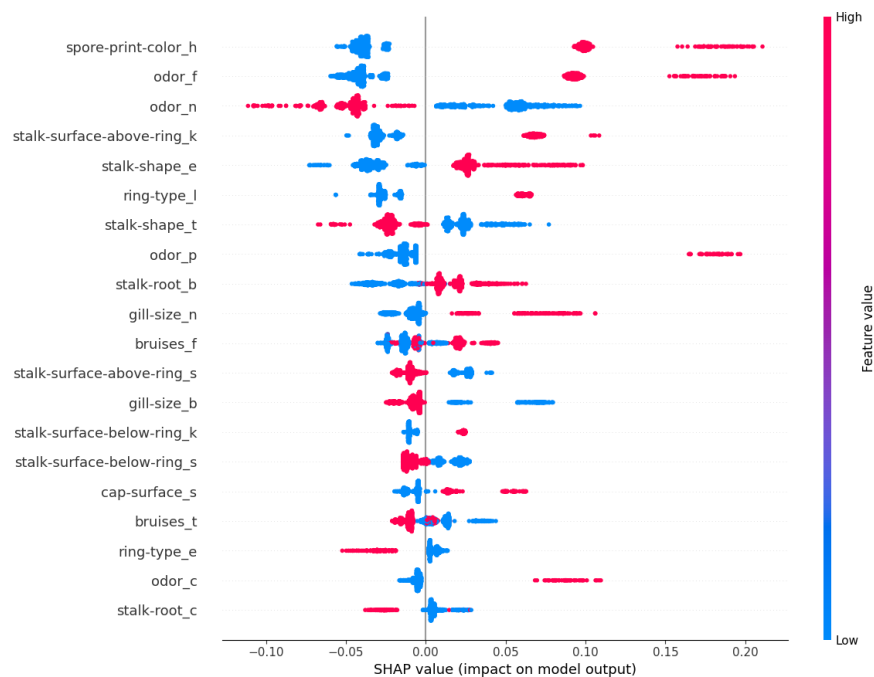


図 14: SHAP(影響方向と大きさ)

4. 考察

ランダムフォレストは全ての評価指標において 1.00 を示し、テストデータで誤分類を起こしていない事が分かる。PDP/ICE プロットおよび SHAP 解析から、「無臭＝食用」という強力で一貫したルールがデータセット内でほぼ例外なく成立しており、モデルはこの規則を優先的に学習したことが考えられる。また、ランダムフォレストは多数の決定木を組み合わせ、階層的に「無臭かどうか」をまず判断し、その後に odor_f (悪臭) や spore-print-color などの補助的特徴を使うことで、極めて安定した分類を実現したと考えられる。

問題 3

ai_13.ipynb で示した β -セクレターゼ(BACE1)の IC_{50} の物性値を予測する順伝播型ニューラルネットワークは、予測値と実測値のズレが若干大きいことが確認できた。そこで、別の回帰手法を用いてより正確な予測モデルを構築し、予測精度が向上した要因について議論せよ。

1. はじめに

各データセットから説明変数と目的変数を抽出し、`y.ravel()`メソッドを用いて、目的変数の形状をモデルが扱いやすい1次元配列に変換した。

2. モデルの訓練・検証

データセットをランダムフォレストに学習させた。今回使用したランダムフォレストの決定木の数は100個である。以下に作成したプログラムを示す。

```
import numpy as np
import matplotlib.pyplot as plt
import deepchem as dc
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error,
r2_score

featurizer = dc.featurizer.RDKitDescriptors()
tasks, datasets, transformers =
dc.molnet.load_bace_regression(featurizer)
train_set, val_set, test_set = datasets

X_train, y_train = train_set.X, train_set.y.ravel()
X_val, y_val = val_set.X, val_set.y.ravel()
X_test, y_test = test_set.X, test_set.y.ravel()

model = RandomForestRegressor(n_estimators=100,
random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE: {mse:.4f}")
print(f"Test R^2: {r2:.4f}")

plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.xlabel('True normalized pIC50')
plt.ylabel('Predicted normalized pIC50')
plt.title(f'RandomForest Test MSE={mse:.3f},
R2={r2:.3f}')
plt.plot([y_test.min(), y_test.max()],
[y_test.min(), y_test.max()], 'r--')
plt.tight_layout()
plt.show()

```

3. 結果

予測値と実測値の間の平均二乗誤差（MSE）を計算した。また、決定係数（ R^2 ）を計算した。テストデータの実測値を横軸に、予測値を縦軸にとった散布図を以下に示す。

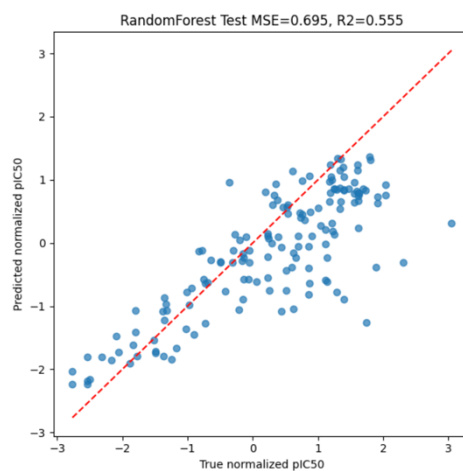


図 15: 散布図

4. 考察

ランダムフォレスト回帰が順伝播型ニューラルネットワーク（FNN）と比較して予測精度が向上する要因は非線形性の捕捉能力/過学習への耐性/特徴量スケーリングや前処理への頑健性の3つだと考えられる。

非線形性の捕捉能力: FNN は多くの学習データと適切なパラメータ最適化が必要であり、データが少ない場合は学習不足または過学習が発生しやすいという制約がある。ランダムフォレストは、複数の決定木を組み合わせることで、特徴量と目的変数間の複雑な非線形関係や、特徴量間の相互作用を非常に効率的に学習可能である。これは、分子の物性値予測のような複雑な関係性を持つデータにおいて特に有効であると考えられる。

過学習への耐性: 多数の決定木をアンサンブルすることで、個々の決定木が持つ過学習の傾向を打ち消し、モデル全体の汎化性能（未知のデータに対する予測能力）を高める。特に、BACE データセットのように、ディープラーニングモデルが真価を発揮するにはやや小規模なデータセットにおいて、ランダムフォレストは安定した性能を発揮しやすいと考えられる。

特徴量スケーリングや前処理への頑健性: FNN は勾配降下法に依存するため、特徴量のスケーリングや標準化が必須である。スケーリングが適切でないと学習が収束しにくく、精度低下や訓練不安定性が発生する。ランダムフォレストは決定木ベースのモデルであるため、特徴量のスケール（値の範囲）や分布にあまり影響されない。